

AI Assisted Coding

Name : Amma Anjali

H.no : 2303A52385

Batch : 33

Course Code : 23CS002PC304

Assignment Number : 15.5

Lab 15 – Backend API Development: Creating RESTful services with AI

Task 1 – Student Attendance API Task:

Create a RESTful API for student attendance tracking.

Instructions:

- Endpoints required: o GET /attendance → View attendance records o POST /attendance → Mark attendance o PUT /attendance/{id} → Update attendance record o DELETE /attendance/{id} → Remove attendance entry
- Ensure JSON-based request and response handling.

Expected Output:

- API for maintaining and updating attendance records.

o GET /inventory → List all items o POST

/inventory → Add new item o PUT

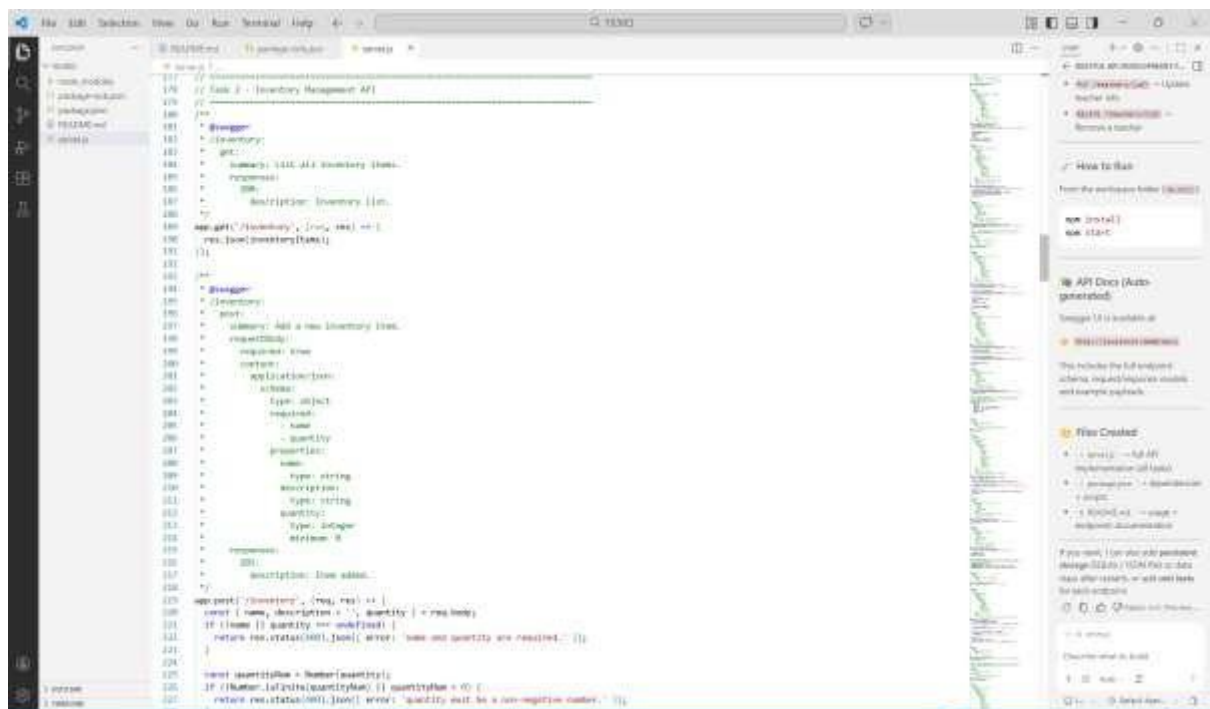
/inventory/{id} → Update stock quantity o

DELETE /inventory/{id} → Remove an item

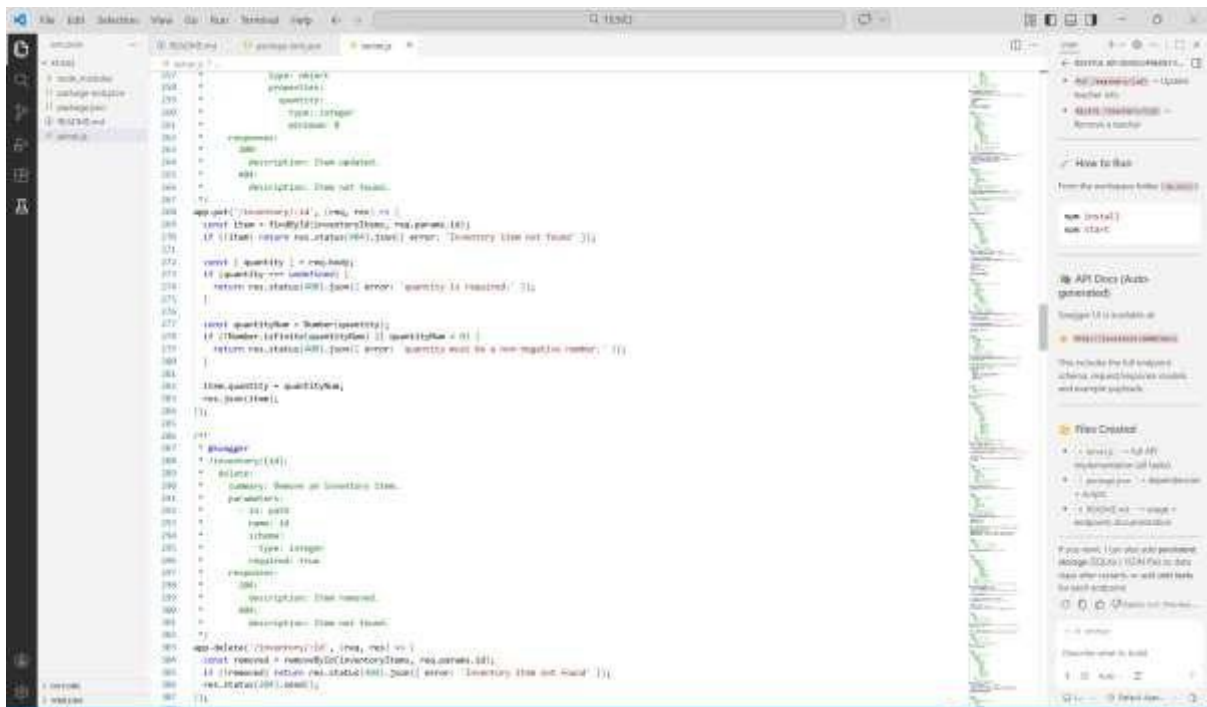
- Implement validation for stock values.

Expected Output:

- API capable of managing inventory stock levels.



```
177 // Task 2 - Inventory Management API
178 //
179 //
180 //
181 // Swagger
182 //
183 // Inventory
184 // GET:
185 // Summary: List all inventory items.
186 // Responses:
187 // 200:
188 // Description: Inventory list.
189 //
190 // GET: /inventory, {req, res} => {
191 //   res.json(inventoryItems);
192 // }
193 //
194 //
195 // Swagger
196 //
197 // Inventory
198 // POST:
199 // Summary: Add a new inventory item.
200 // request body:
201 //   type: object
202 //   required:
203 //     - name
204 //     - quantity
205 //     - description
206 //   properties:
207 //     name:
208 //       type: string
209 //       description:
210 //         type: string
211 //         quantity:
212 //           type: integer
213 //           minimum: 0
214 //   responses:
215 //     201:
216 //       description: Item added.
217 //
218 // POST: /inventory, {req, res} => {
219 //   const { name, description, quantity } = req.body;
220 //   if (!name || quantity == undefined) {
221 //     return res.status(400).json({ error: 'name and quantity are required...' });
222 //   }
223 //
224 //   const quantityAsNumber = Number(quantity);
225 //   if (Number.isNaN(quantityAsNumber) || quantityAsNumber < 0) {
226 //     return res.status(400).json({ error: 'quantity must be a non-negative number...' });
227 //   }
228 // }
```



Task 3 – Hotel Room Booking API Task:

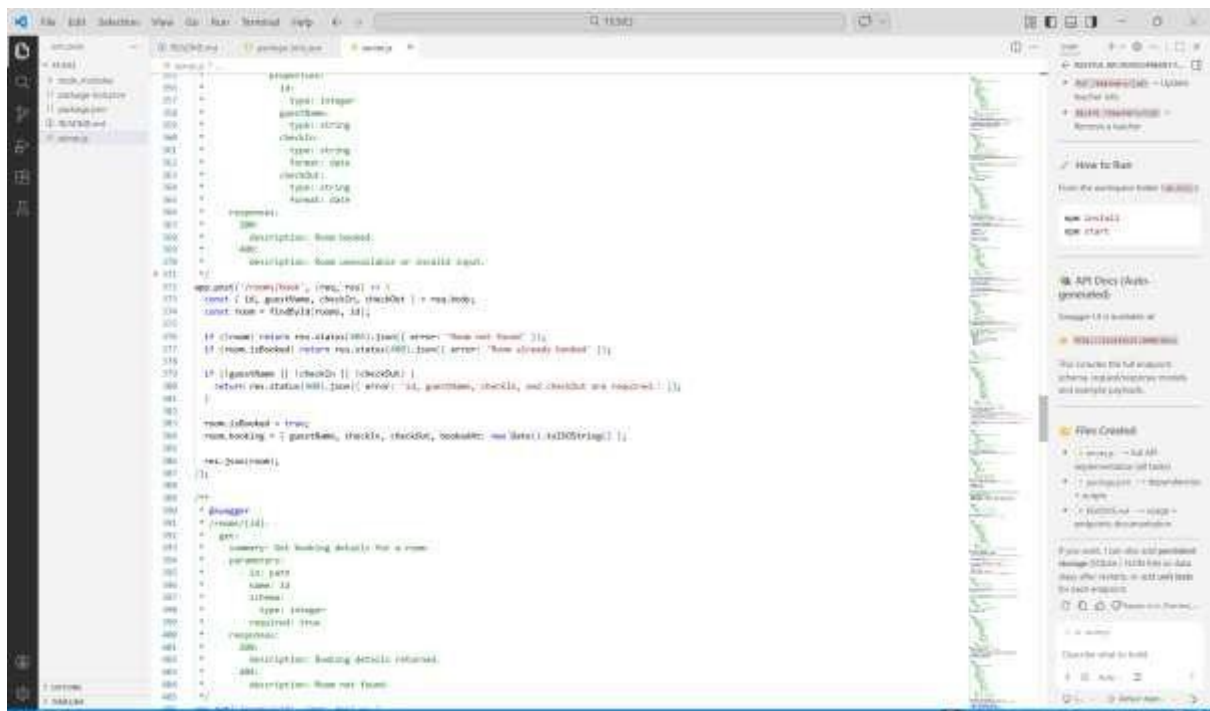
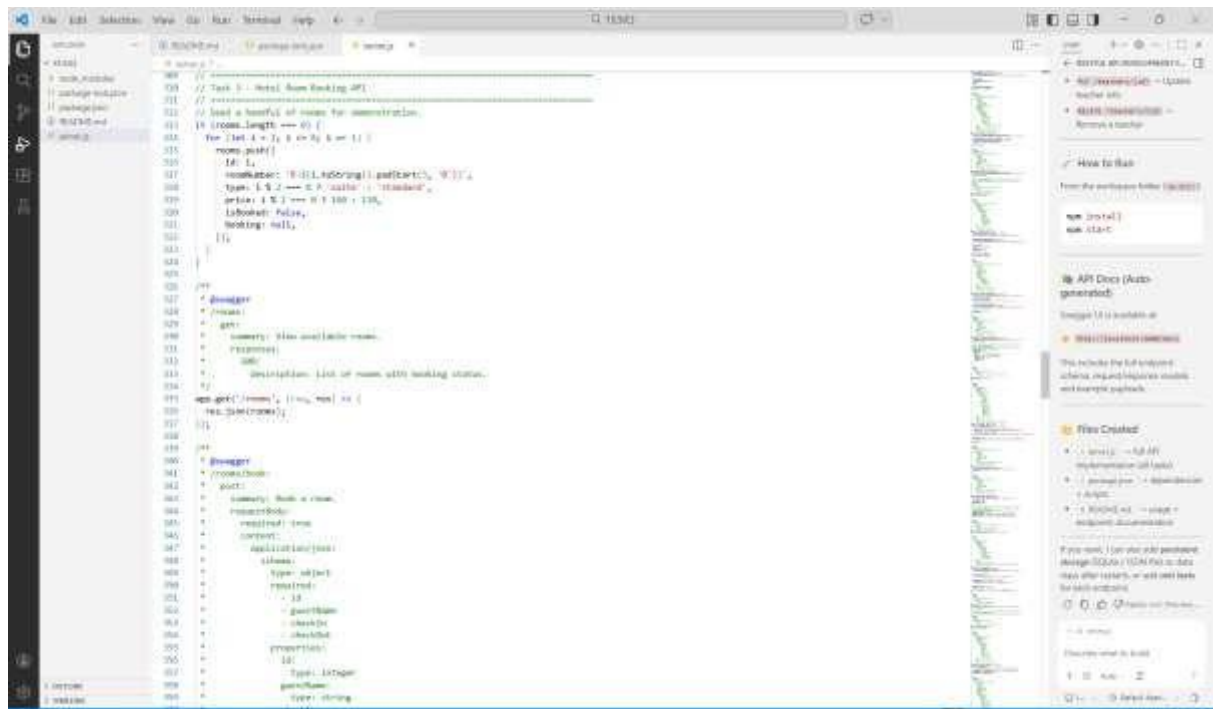
Develop a RESTful API for hotel room booking.

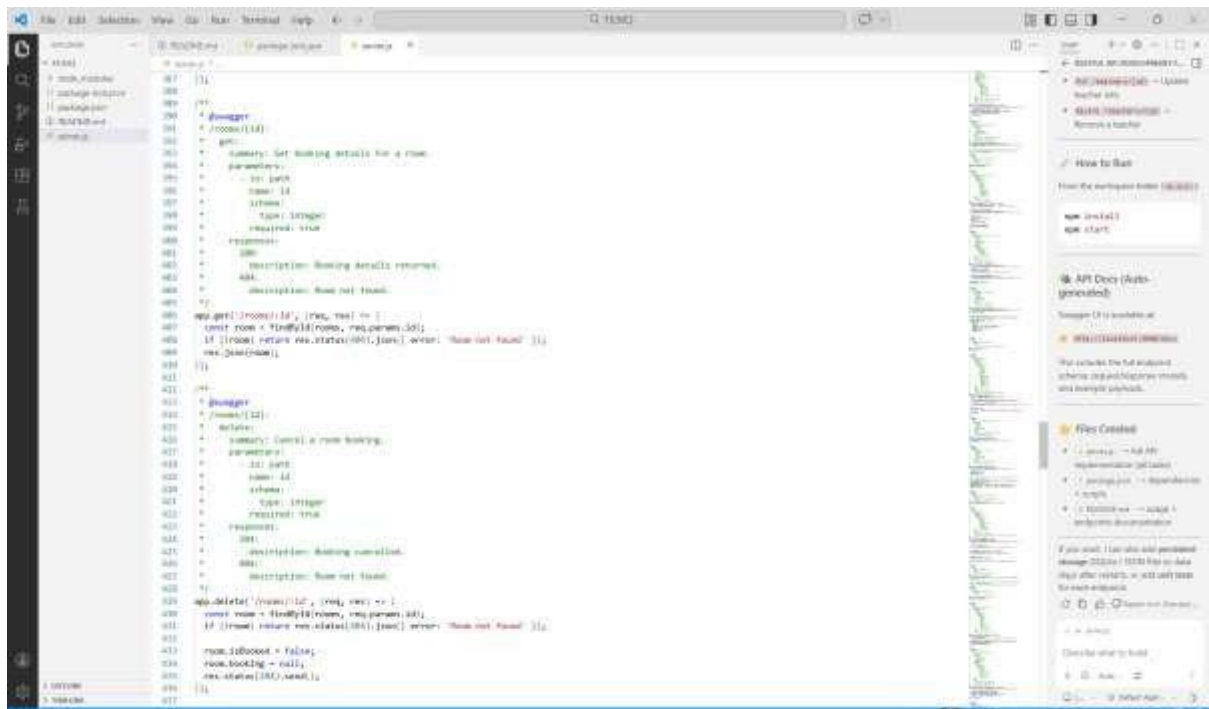
Instructions:

- Endpoints required:
 - o GET /rooms → View available rooms
 - o POST /rooms/book → Book a room
 - o GET /rooms/{id} → View booking details
 - o DELETE /rooms/{id} → Cancel booking
- Implement error handling for unavailable rooms.

Expected Output:

- API simulating hotel room booking operations.





Task 4 – Online Voting API Task:

Create a RESTful API for an online voting system.

Instructions:

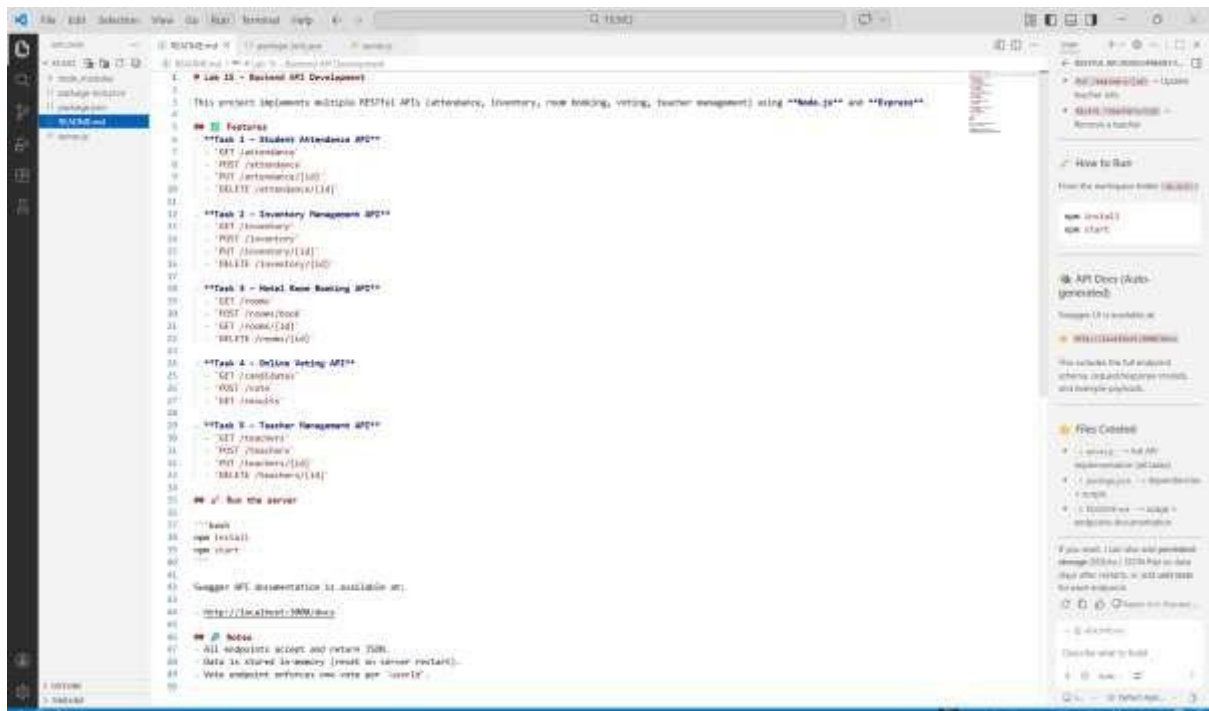
- Endpoints required:
 - o GET /candidates → List candidates
 - o POST /vote → Cast a vote
 - o GET /results → View voting results
- Ensure one vote per user using in-memory validation.

Expected Output:

- API simulating a secure online voting process.

- Functional API for managing teacher information.

[illegible]



Install Project & Run the Project



Server listening on <http://localhost:3000>

Swagger UI available at <http://localhost:3000/docs> <http://localhost:3000/docs> :

Swagger UI interface showing the API documentation for Lab 15 - Backend API Development. The interface displays the default API endpoints and their corresponding HTTP methods.

Lab 15 - Backend API Development

RESTful APIs for attendance, inventory, room booking, voting, and teacher management.

Servers:

default

Method	Endpoint	Description
GET	/attendance	Retrieve all attendance records
POST	/attendance	Mark attendance for a student
PUT	/attendance/{id}	Update an attendance record
DELETE	/attendance/{id}	Delete an attendance record
GET	/inventory	List all inventory items
POST	/inventory	Add a new inventory item
PUT	/inventory/{id}	Update stock quantity for an item

Swagger UI interface showing the details for the POST /attendance endpoint. The interface displays the request body and the response.

POST /attendance

Mark attendance for a student

Parameters: No parameters

Request body:

Body Value: Schema

```
{  "studentName": "Harish",  "date": "2024-02-15",  "status": "present"}
```

Buttons: Execute, Clear

Responses: Out

Responses

Curl

```
curl -X POST \
  http://localhost:3000/attendance \
  -H 'Accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "studentName": "Harish",
    "date": "2025-03-13",
    "status": "present"
  }'
```

Request URL

http://localhost:3000/attendance

Server response

Code Details

201

Response body

```
{
  "id": 1,
  "studentName": "Harish",
  "date": "2025-03-13",
  "status": "present"
}
```

Response headers

```
connection: keep-alive
content-length: 78
content-type: application/json; charset=utf-8
date: Fri, 13 Mar 2025 05:47:41 GMT
etag: W/"48-wu72188Czjy8h3m4u8t8"
keep-alive: timeout=5
powered-by: Express
```

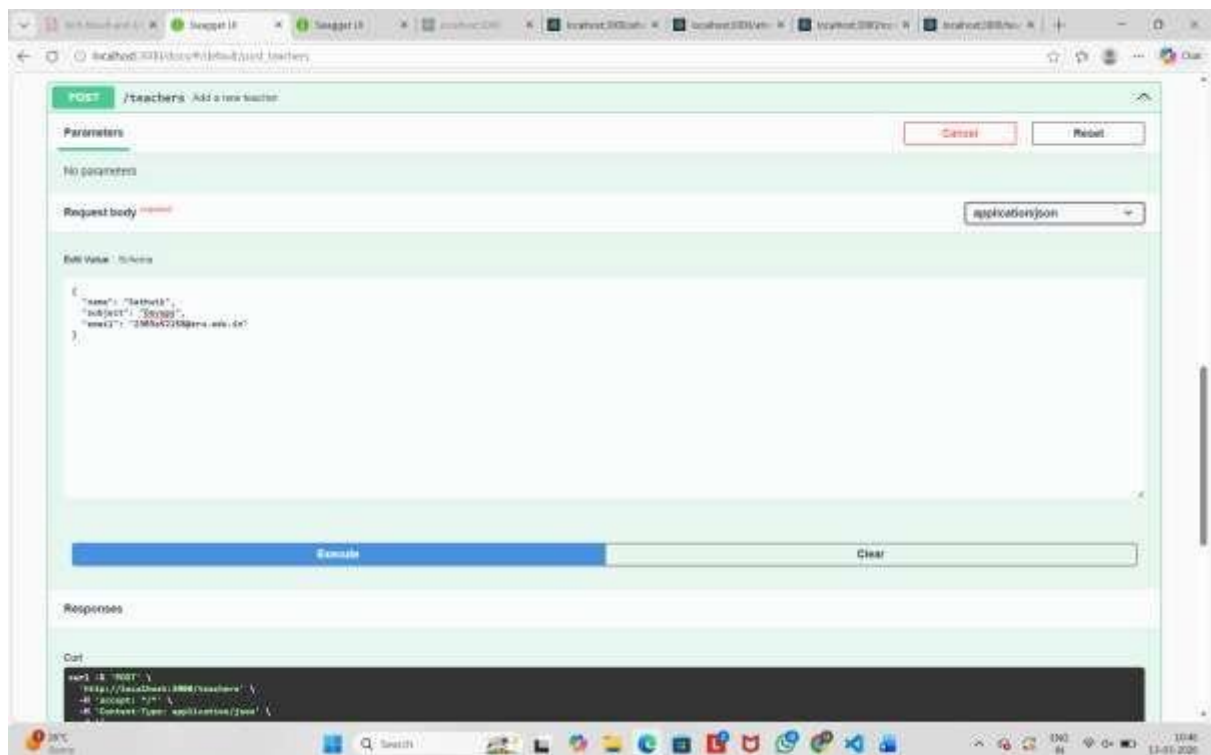
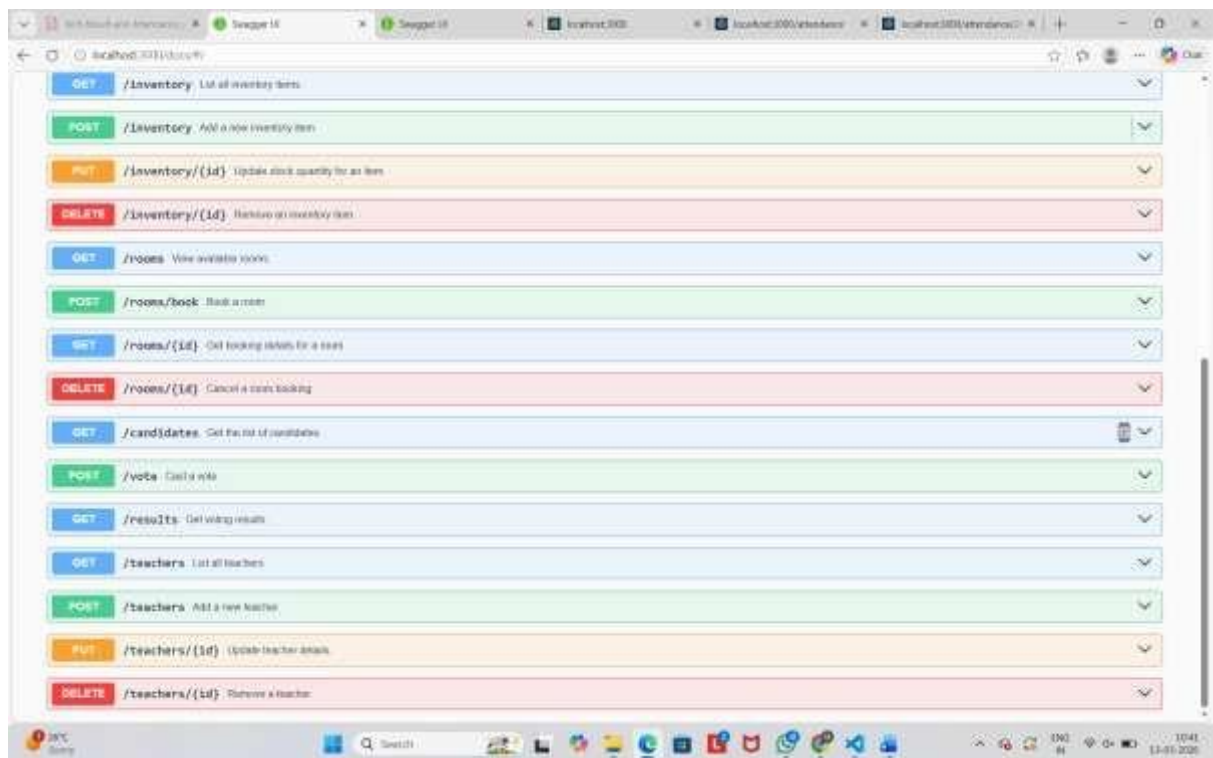
Responses

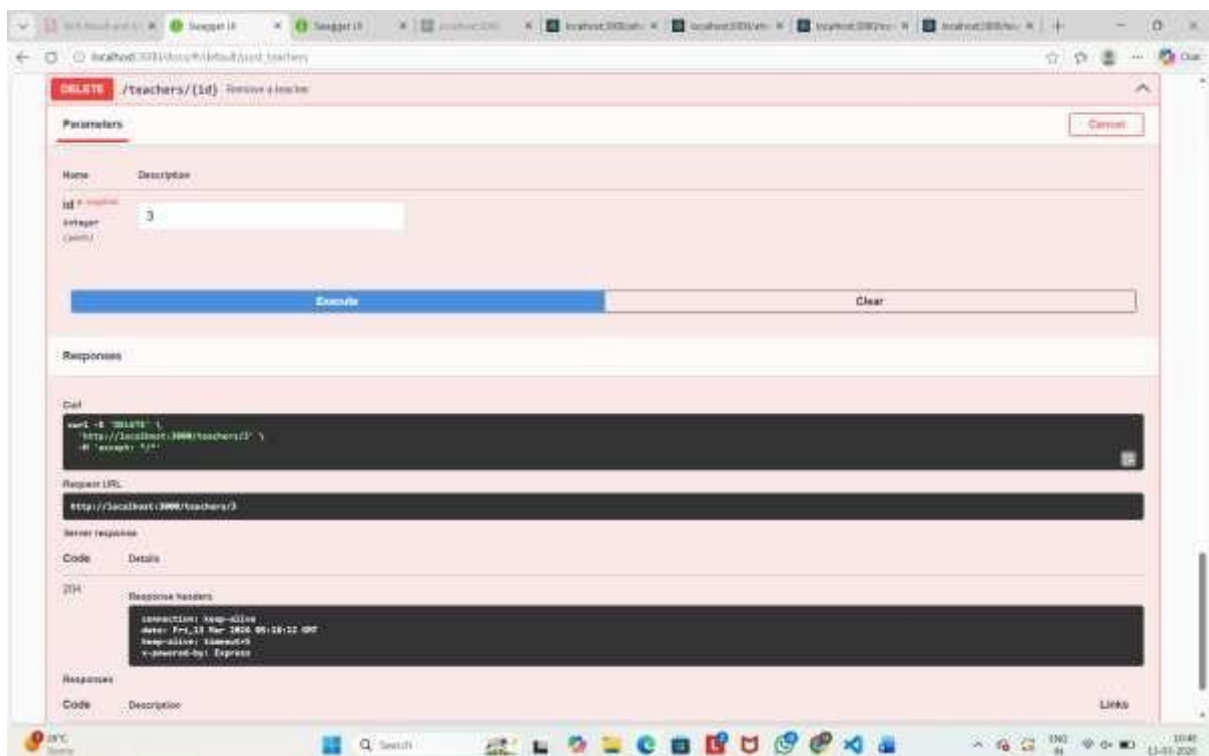
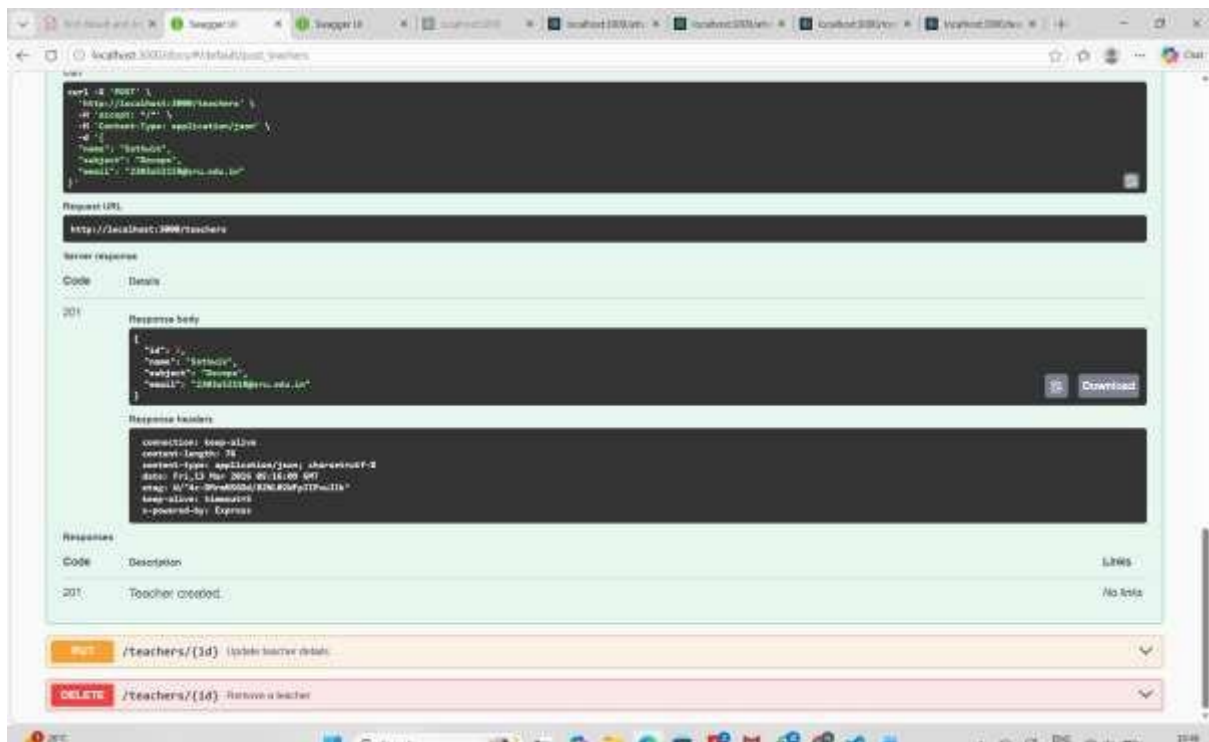
Code	Description	Links
201	Attendance recorded.	No links

PUT /attendance/[id] Update an attendance record

Front-end

```
{
  "id": 1,
  "studentName": "string",
  "date": "2025-03-13",
  "status": "present"
},
{
  "id": 2,
  "studentName": "string",
  "date": "2025-03-13",
  "status": "present"
},
{
  "id": 3,
  "studentName": "Harish",
  "date": "2025-03-13",
  "status": "present"
}
```





GITHUB LINK : <https://github.com/2303A52387/AI-ASSISTANT-CODING/blob/main/15.zip>